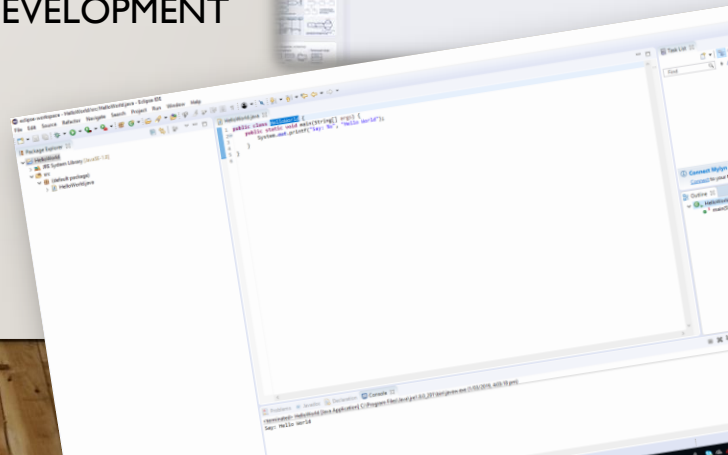
THE UNIVERSITY OF AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

OBSERVER PATTERN

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

PARAMVIR SINGH



AGENDA

- Q/A Discussion
  - Template Pattern lecture exercises
- Pattern of the Day
  - Observer Pattern

Q/A DISCUSSION – TEMPLATE PATTERN

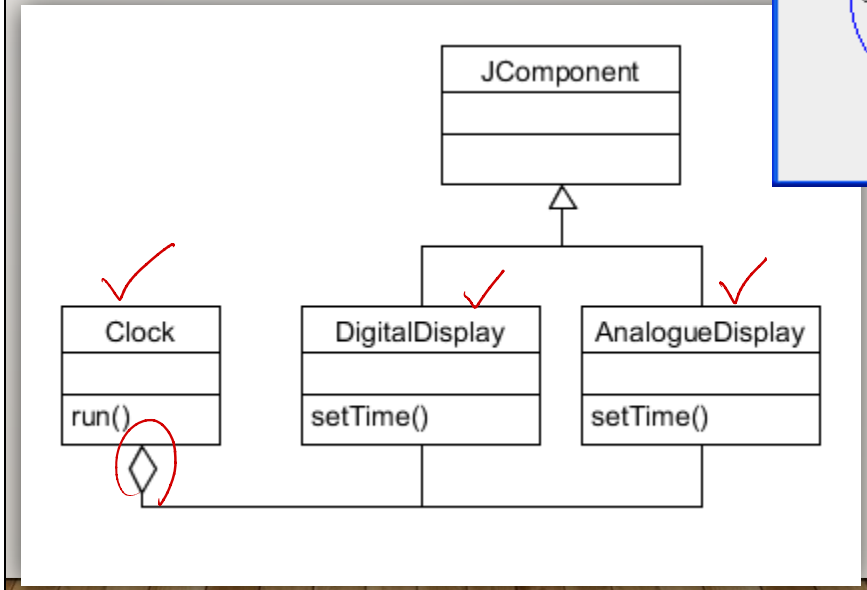
- **Template pattern in Java API?**
  - Classes AbstractList, AbstractSet, etc.
- **Design objectives addressed by template method pattern?**
  - Higher flexibility
    - we can provide additional implementations to the hook steps in subclasses without needing to change the client or the rest of the inheritance hierarchy
    - Eliminates duplicate code, which is great for flexibility
  - Higher robustness
    - template method is the only entry point to the algorithm/operation execution
    - Other classes can not directly access the hooks meant to complete the operation promised by the template method
- **SOLID principles followed by template pattern?**
  - Follows SRP, OCP, and LSP

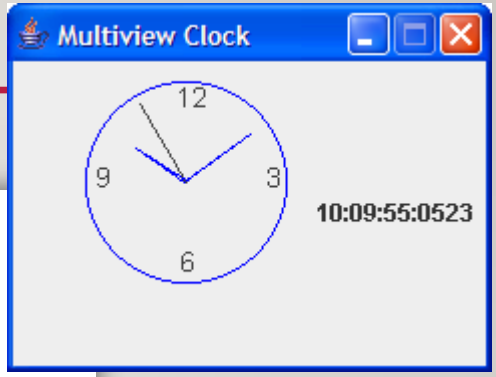
DESCRIPTION

- ***Context***:
  - When an association (especially when *bi-directional*) is created between two classes, the code for the classes becomes inseparable.
  - If you want to reuse one class, then you also have to reuse the other.
- ***Problem***:
  - Given two classes, where objects of one class are interested in the state of an object of another class and wants to get notified whenever there is any change.
  - How do you reduce the interconnection between such classes, especially between classes that belong to different modules or subsystems?
- ***Forces***:
  - You want to maximize the flexibility of the system to the greatest extent possible

CLOCK APPLICATION

```
public class Clock {
   private AnalogueDisplay _analogue;
   private DigitalDisplay _digital;

   public Clock() {
      _analog = new AnalogueDisplay();
      _digital = new DigitalDisplay();
      // Code to construct GUI …
   }

   public void run() {
      repeat every second:
         _analogueDisplay.setTime( … );
         _digitalDisplay.setTime( … );
   }
}
```

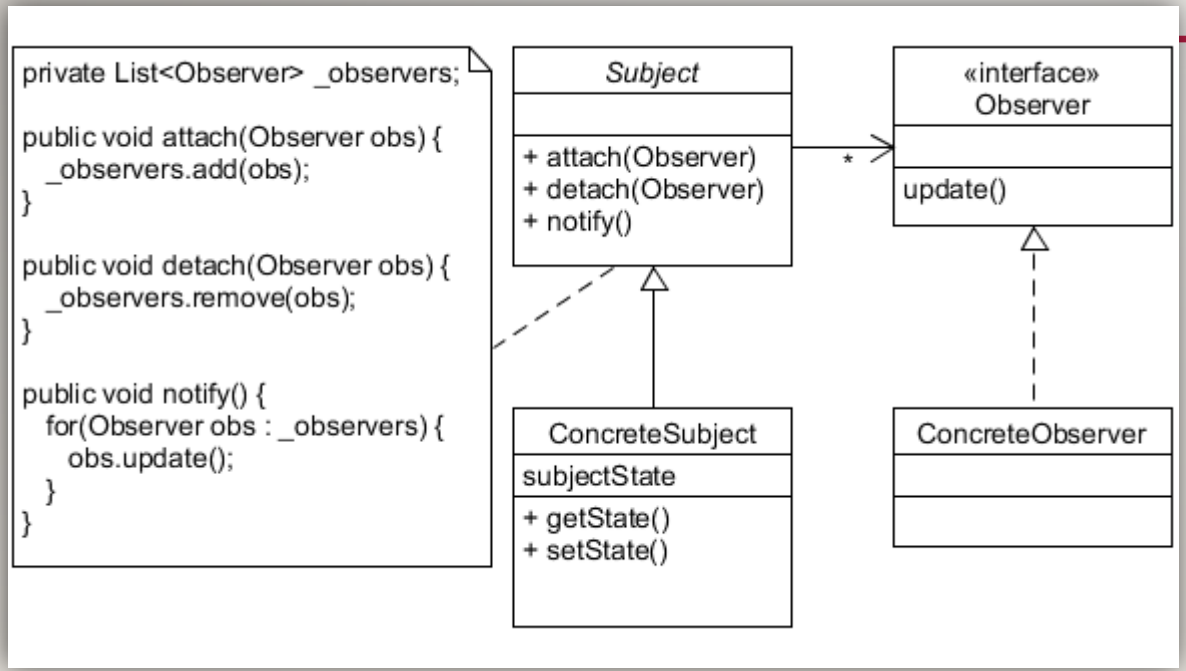


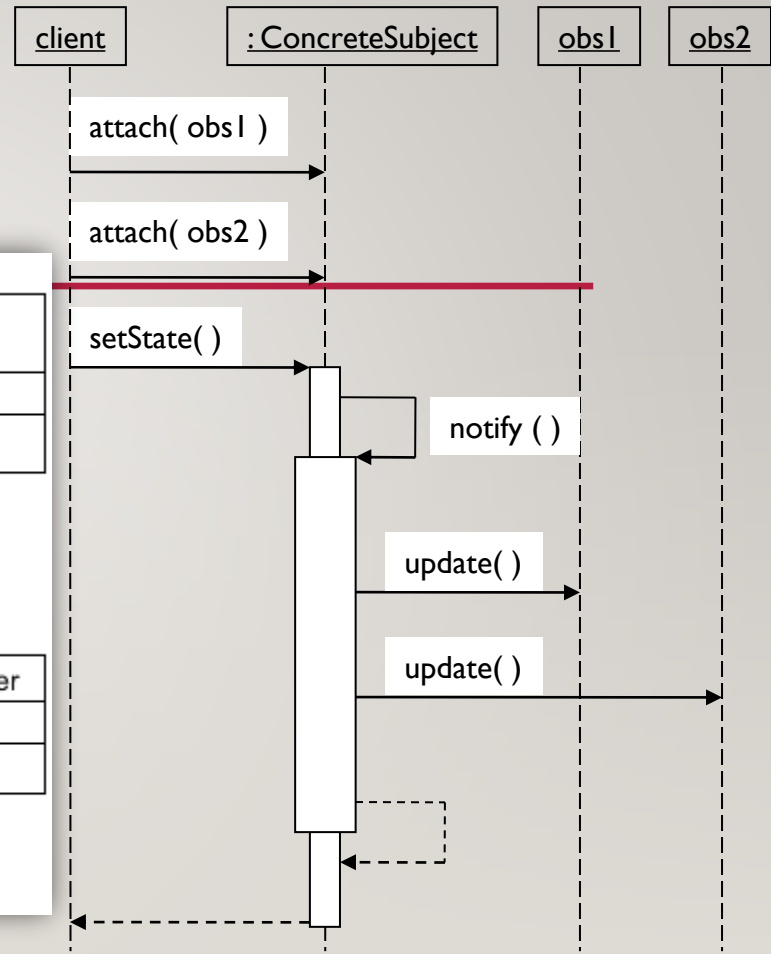


THE OBSERVER PATTERN

- ***Solution:***

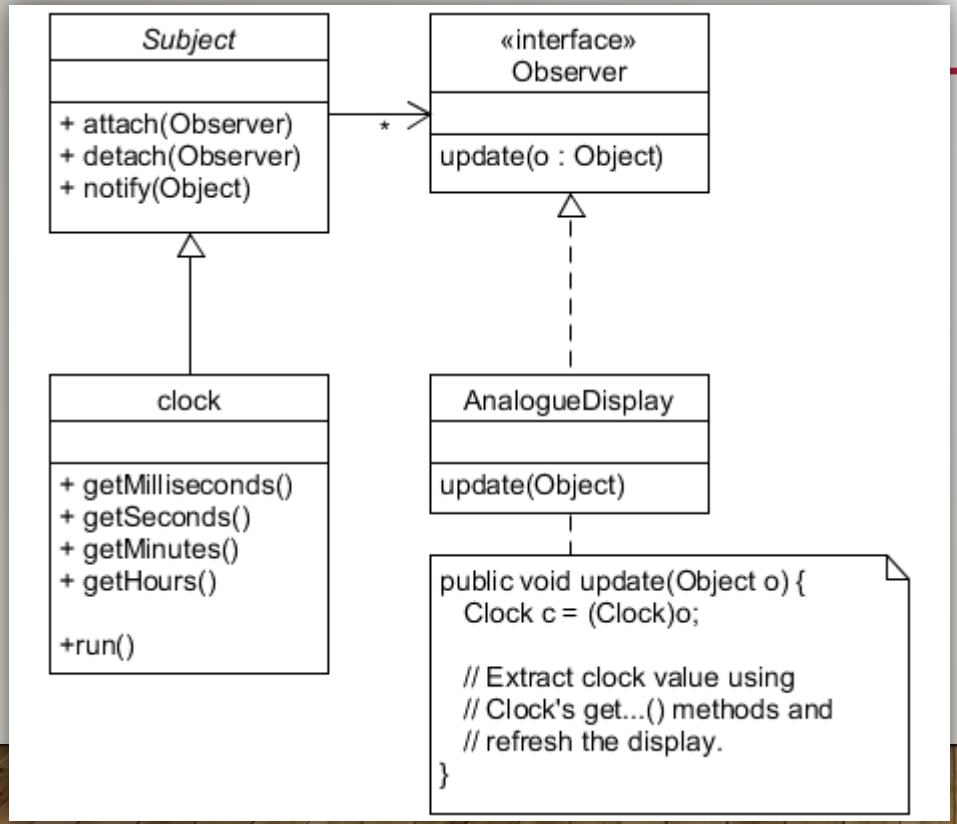
```
private List<Observer> _observers;

public void attach(Observer obs) {
   _observers.add(obs);
}

public void detach(Observer obs) {
   _observers.remove(obs);
}

public void notify() {
   for(Observer obs : _observers) {
      obs.update();
   }
}
```

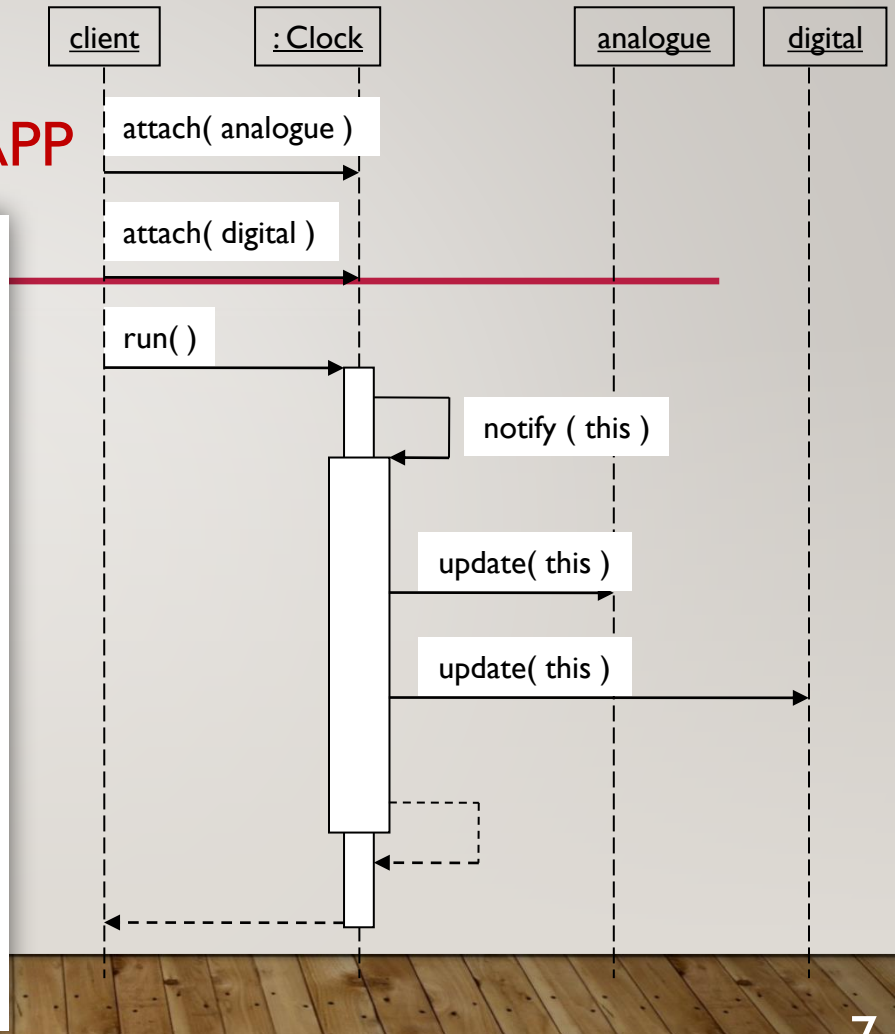




See Observer and Observable from package java.util.

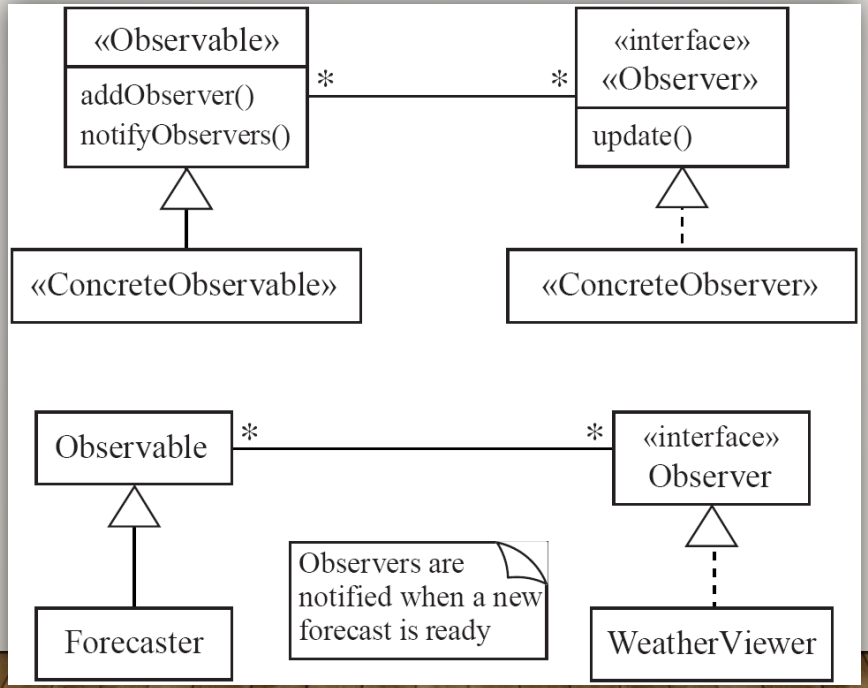
THE OBSERVER PATTERN
APPLICATION TO CLOCK APP



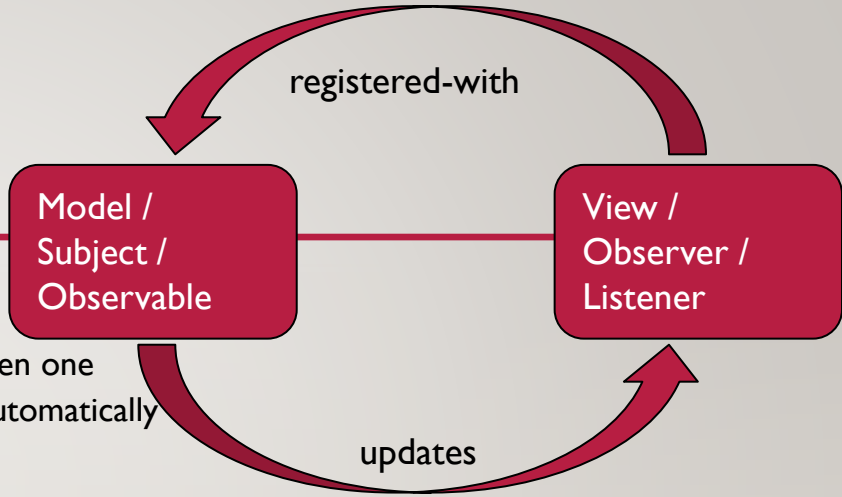


THE OBSERVER PATTERN
SUMMARY CLASS DIAGRAM

- ***In Brief:***



APPLICATION



- **Intent**
  - Define a one-to-many dependency between objects so that when one object changes state, all dependents are notified and updated automatically
- **Also known as**
  - Publish-Subscribe
- **Applicability** - use the Observer pattern in any of the following situations:
  - When an abstraction has two aspects, one dependent on the other, encapsulating these aspects in separate objects lets you vary and reuse them independently
  - When a change to one object requires changing others, and you don't know how many objects need to be changed
  - When an object should be able to notify other objects without making assumptions about who these objects are

ANTI-PATTERNS

- Connect an observer directly to an observable so that they both have references to each other.
- Make the observers *subclasses* of the observable.

REVIEW

- Learnt something about observer pattern:
  - Rationale behind observation pattern
  - Observer pattern is basic to event handling mechanisms
  - Observer pattern makes good use of OCP and DIP principles

SUGGESTED TASKS

- Study the implementation of **Observer Demo** example code (Canvas)
- Which **design objectives** are helped by the **design pattern** studied in today's class?
- Can you see **some instances of SOLID principles** being followed (or not followed) – in the examples of this design pattern?
- Are there instances of template pattern implemented in the Java API?

REFERENCES

- **[Books]**
  - Chapter 6 – Design Patterns (**Canvas**), Object-Oriented Software Engineering: Practical Software Development using UML and Java by Timothy Lethbridge and Robert Laganiere
  - Design Patterns: Elements of Reusable Object-Oriented Software by John Vlissides, Ralph Johnson, Richard Helm, Erich Gamma
  - Agile Software Development, Principles, Patterns, and Practices by Robert C Martin
- **[Media]**
  - A JavaWorld introductory article on design patterns
    - http://www.javaworld.com/javaworld/jw-10-2001/jw-1012-designpatterns.html
- **[See more on Canvas Lecture page]**